COMP7710

Programming Fundamentals

Today's lecture

JDK, IDEs and jshell

Course Convenor

Bernardo Pereira Nunes

Australian National University

All lectures are recorded and made available on Canvas

Agenda

> What is JDK?
> Compiling a program using javac
> IDEs
> IntelliJ Demo
> jshell
> Utilities
> Exercises

Java Development Kit (JDK)

> The JDK is a development environment for building applications and components using the Java programming language.

> It includes tools useful for developing, testing, and monitoring programs written in Java and running in the Java platform.

> The JDK includes:

>> Java Compiler (javac) → converts java source code (.java) into **bytecode (.class)** → runs in any machine with a JVM

>> Java Virtual Machine (JVM) → executes Java bytecode, handles memory management, security checks, and runtime optimization; responsible for making Java platform-independent; the JVM itself is, however, platform-dependent

>> Java Runtime Environment (JRE)

JRE is a software layer built on top of the operating system that provides the class libraries and runtime resources necessary for executing Java programs.

Source: https://www.oracle.com/java/technologies/javase/jdk25-readme-downloads.html

Java Development Kit (JDK)

> The JDK includes:

>> …

>> Standard Libraries (Java API) → pre-written code for common tasks: data structures (Collections Framework), I/O File, Networking, Concurrency, Graphics, Logging, etc.

>> Development Tools → jshell (Java's interactive shell), javadoc (documentation generator), debugging and monitoring tools, etc:

Java® Development Kit Version 25 Tool Specifications

All Platforms

- jar - create an archive for classes and resources, and manipulate or restore individual classes or resources from an archive
- jarsigner - sign and verify Java Archive (JAR) files
- java - launch a Java application
- javac - read Java class and interface definitions and compile them into bytecode and class files
- javadoc - generate HTML pages of API documentation from Java source files
- javap - disassemble one or more class files
- jcmd - send diagnostic command requests to a running Java Virtual Machine (JVM)
- jconsole - start a graphical console to monitor and manage Java applications
- jdb - find and fix bugs in Java platform programs
- jdeprscan - static analysis tool that scans a jar file (or some other aggregation of class files) for uses of deprecated API elements
- jdeps - launch the Java class dependency analyzer
- jfr - parse and print Flight Recorder files
- jhsdb - attach to a Java process or launch a postmortem debugger to analyze the content of a core dump from a crashed Java Virtual Machine (JVM)
- jinfo - generate Java configuration information for a specified Java process
- jlink - assemble and optimize a set of modules and their dependencies into a custom runtime image
- jmap - print details of a specified process
- jmod - create JMOD files and list the content of existing JMOD files
- jnativescan - static analysis tool that scans one or more jar files for uses of native functionalities
- jpackage - package a self-contained Java application
- jps - list the instrumented JVMs on the target system
- jrunscript - run a command-line script shell that supports interactive and batch modes
- jshell - interactively evaluate declarations, statements, and expressions of the Java programming language in a read-eval-print loop (REPL)
- jstack - print Java stack traces of Java threads for a specified Java process
- jstat - monitor JVM statistics
- jstatd - monitor the creation and termination of instrumented Java HotSpot VMs
- jwebserver - launch the Java Simple Web Server
- keytool - manage a keystore (database) of cryptographic keys, X.509 certificate chains, and trusted certificates
- rmiregistry - create and start a remote object registry on the specified port on the current host
- serialver - return the `serialVersionUID` for one or more classes in a form suitable for copying into an evolving class

Windows Only

- jabswitch - enable or disable Java Access Bridge
- jaccessinspector - examine accessible information about the objects in the Java Virtual Machine using the Java Accessibility Utilities API
- jaccesswalker - navigate through the component trees in a particular Java Virtual Machine and present the hierarchy in a tree view
- javaw - launch a Java application without a console window
- kinit - obtain and cache Kerberos ticket-granting tickets

Source: https://www.oracle.com/java/technologies/javase/jdk25-readme-downloads.html

Java Development Kit (JDK)

> The JDK includes:

>> Java Compiler (javac)

>> Java Virtual Machine (JVM)

>> Java Runtime Environment (JRE)

>> Standard Libraries (Java API)

>> Development Tools

Note that **arm64, x86, x64** is about **CPU architecture**, not the Operating System (Linux, Windows, macOS).

Different CPUs understand different machine instructions:
> **x86 / x64**: Intel & AMD processors
> **ARM64 (AArch64)**: Apple Silicon (M1/M2/M3), many servers, mobile devices

A JVM compiled for x86 cannot run on ARM, and vice versa.

Write once, run anywhere **(with the right JVM)**

Download at:

https://www.oracle.com/java/technologies/downloads/

Linux **macOS** **Windows**

Product/file description	File size
ARM64 Compressed Archive	198.31 MB
ARM64 DMG Installer	197.77 MB
x64 Compressed Archive	200.50 MB
x64 DMG Installer	199.96 MB

How do I know my CPU architecture?

```
Using terminal:
Linux/macOS: uname -m
Windows: echo %PROCESSOR_ARCHITECTURE%
You can also ask ChatGPT, DeepSeek, …
```


Getting Started: launching a Java program from the command line

```
bpn@Bernardos-MBP Week_02 % java Welcome.java
Welcome to Core Java!
====================
bpn@Bernardos-MBP Week_02 %
```

```
java Welcome.java
```

```
bpn@Bernardos-MBP Week_02 % ls
02_Intro_to_Java.pptx   Welcome.java            ~$02_Intro_to_Java.pptx
bpn@Bernardos-MBP Week_02 % java Welcome.java
Welcome to Core Java!
====================
bpn@Bernardos-MBP Week_02 % java Welcome
Error: Could not find or load main class Welcome
Caused by: java.lang.ClassNotFoundException: Welcome
bpn@Bernardos-MBP Week_02 % javac Welcome.java
bpn@Bernardos-MBP Week_02 % java Welcome
Welcome to Core Java!
====================
bpn@Bernardos-MBP Week_02 %
```

```
javac Welcome.java
java Welcome
```

In older versions of Java, programs were compiled using `javac` and executed using `java`, allowing code to be compiled once and run multiple times.

Now, let's run `ImageViewer.java` and take a look at its code!

Integrated Development Environment (IDE)

An **IDE** is a software application that provides all the tools needed to **develop, run, and debug** (in our case) **Java programs** in one place

Examples of IDEs for Java
Common IDEs used for Java development: **Eclipse**, **IntelliJ IDEA**, **NetBeans**

↘ used in this course (including the final exam!)

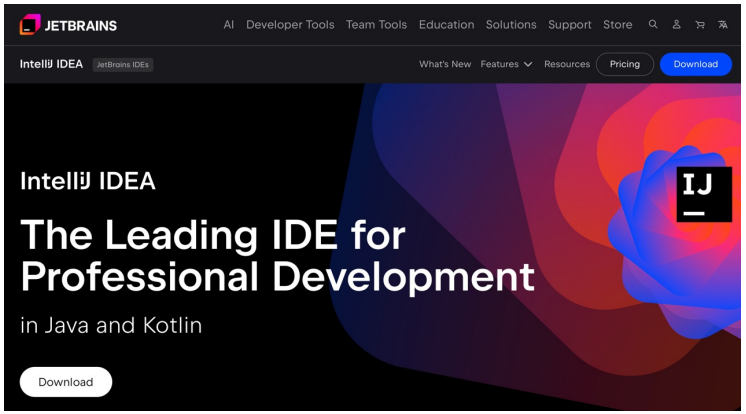
(More in COMP6442: Software Construction)

← **IDE can help us with:**

> Writing Java code efficiently (simplifies the coding process through their features (e.g., code completion))
> Compiling code using javac & Running programs on the JVM (code compilation and execution via their interface)
> Debugging Tools (breaking points, variable inspection and tracking, …)
> Version Control Integration (e.g.,Git, SVN) (manage source code versions, track changes, and collaborate)
> Managing large projects with many classes (organises packages, classes; handles dependencies, …)

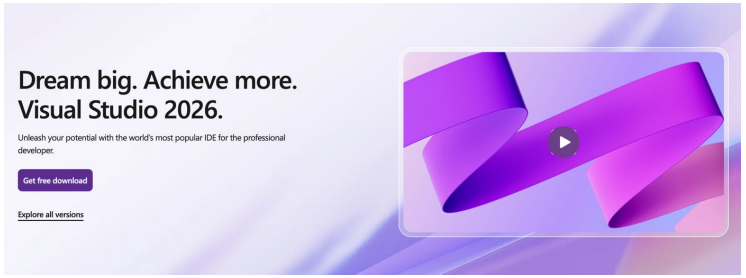
The IDE's Code Editor can help with syntax highlighting for Java, code completion, automatic refactoring, detecting errors as you type, suggesting fixes automatically, reducing boilerplate code.

Integrated Development Environment (IDE)

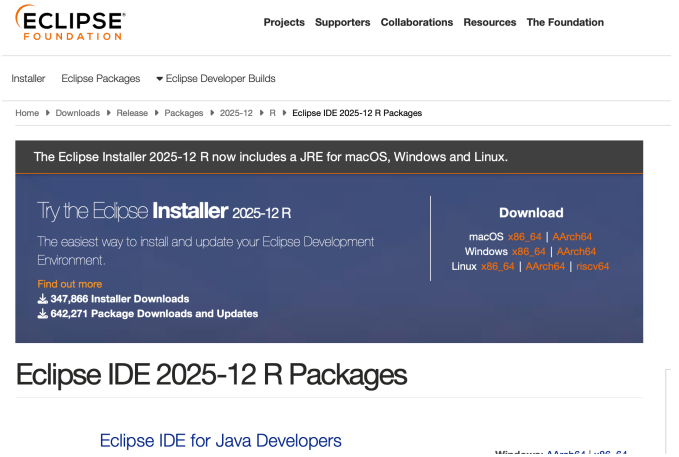


https://www.jetbrains.com/idea/download/

Note that IntelliJ IDEA has two editions: Community Edition and Ultimate. You must select the Community Edition (no subscription required). IntelliJ IDEA now uses a single, unified installer.



https://visualstudio.microsoft.com/#vs-section



https://www.eclipse.org/downloads/packages/installer

https://www.jetbrains.com/help/idea/2025.2

INTELLIJ IDEA | QUICK DEMO

Let's explore the interface now:
accessibility, file, edit, view, …

> Creating our first Java Project:
New Project > Java (Select JDK)

> Project Structure
File > Project Structure; or,
Right click on the project folder > Open Module Settings

> Create a new Class
Right click on the "src" folder > New > Java Class

> Running our Java Program

> Debugging our Java Program

> Compiling using the IDE and javac (using terminal)
javac -d ../out/production/project_name Test.java

destination directory for compiled .class files

source file to compile

JSHELL I excellent way to experiment

> The **JShell** program provides a "read-evaluate-print loop" (REPL).

> You type a Java expression; JShell evaluates your input, prints the result, and waits for your next input. It is much faster than writing a complete program in an integrated development environment.

>Java program development typically involves the following process:
>> Write a complete program
>> Compile it and fix any errors
>> Run the program
>> Figure out what is wrong with it
>> Edit it
>> Repeat the process

JShell helps you try out code and easily explore options as you develop your program. It doesn't replace an IDE. You can test individual statements, try out different variations of a method, and experiment with unfamiliar APIs within a JShell session.

Source for this demo: https://docs.oracle.com/en/java/javase/25/jshell/introduction-jshell.html

JSHELL | excellent way to experiment

JShell accepts:

- > Java statements
- > variables
- > methods
- > class definitions
- > imports
- > expressions

They are called "snippets" and immediately evaluated in JShell

> Go to terminal and type `jshell` or go to IntelliJ > Tools > JShell

```
% jshell
jshell> var counter = 0;
jshell> /exit
```

JShell auto-completes semicolons for complete snippets
IntelliJ requires them explicitly

```
%jshell -v
```

-v enables verbose mode. Repeat the declaration above and compare the results

JSHELL | excellent way to experiment

> What if we want to evaluate the following expression?

```
jshell> 3 + 9
$1 ==> 12
|  created scratch variable $1 : int
```

> We can reuse scratch variables:

```
jshell> 5 + $1
$2 ==> 17
|  created scratch variable $2 : int
```

> We can also store results in a variable: "`var outcome = 5 + $1`"

```
> "COMP7710 uses Java".len <press tab>
> Math. <tab>
```


JSHELL | excellent way to experiment

> What if a snippet is too long? JShell displays a continuation prompt (`...>`)

> Let's copy and paste the following snippet into Jshell:

```
% String concat(String a, String b) {
    return "The result is: " + a + b;
  }
```

> A variable, method, or class can be redefined by entering a new definition, which overrides the previous one

```
% String concat(String a, String b) {
    return a + b;
  }
```

> JShell shows feedback indicating that your snippet was modified

JSHELL | excellent way to experiment

```
jshell> /set feedback
concise    normal     silent     verbose

<press tab again to see synopsis>
jshell> /set feedback
Set the feedback mode describing displayed feedback for entered snippets and commands

<press tab again to see full documentation>
jshell> /set feedback
Set the feedback mode describing displayed feedback for entered snippets and commands:

        /set feedback [-retain] <mode>

Retain the current feedback mode for future sessions:

        /set feedback -retain

Show the feedback mode and list available modes:

        /set feedback

Where <mode> is the name of a previously defined feedback mode.
You may use just enough letters to make it unique.
User-defined modes can be added, see '/help /set mode'

When the -retain option is used, the setting will be used in this and future
runs of the jshell tool.

The form without <mode> or -retain displays the current feedback mode and available modes.
```

Changing the Level of Feedback

JSHELL | excellent way to experiment

> JShell allows method definitions to reference methods, variables, or classes that are not yet defined, supporting exploratory programming.

```
% double volume(double radius) {
    return 4.0 / 3.0 * PI * cube(radius);
  }
```

> Since `PI` was not defined, JShell will warn you about it:

```
|  created method volume(double), however, it cannot be invoked until variable PI,
and method cube(double) are declared

jshell> volume(2.5)
|  attempted to call method volume(double) which cannot be invoked until variable
PI, and method cube(double) are declared
```

→ We can redefine the volume (sphere) method to use Math.PI, or declare a PI variable
Note that we still need to declare `cube(…)`, can we use the `Math.<tab>` library?

JSHELL | excellent way to experiment

```
jshell> "COMP7710 is GREAT!". <tab>
jshell> System. <tab>
jshell> System.exit(0)
jshell> ...
```


JSHELL | excellent way to experiment

> `/help intro` explains what jshell is

> Use `/list` to display a list of snippets and their IDs. If no option is entered, then all active snippets are displayed, but startup snippets aren't.

> Use `/edit` to open an editor. If no option is entered, then the editor opens with the active snippets. Use `/edit -all` to open the editor with all snippets (including failed ones).

> Use `/history` to display what was entered in this and previous sessions. Use `<Ctrl+R>` and `<Ctrl+S>` to search backward or forward in the history.

> Use `/methods` to display information about the methods that were entered.

> Use `/exit` to exit the tool.

JSHELL | excellent way to experiment

> A JShell script is a sequence of snippets and JShell commands in a file, one snippet or command per line.

> Let's go to the `basics.jsh` example

> Load the script in a jshell session:

```
> jshell --startup basics.jsh (OR jshell basics.jsh)
```

> You can create your own scripts:

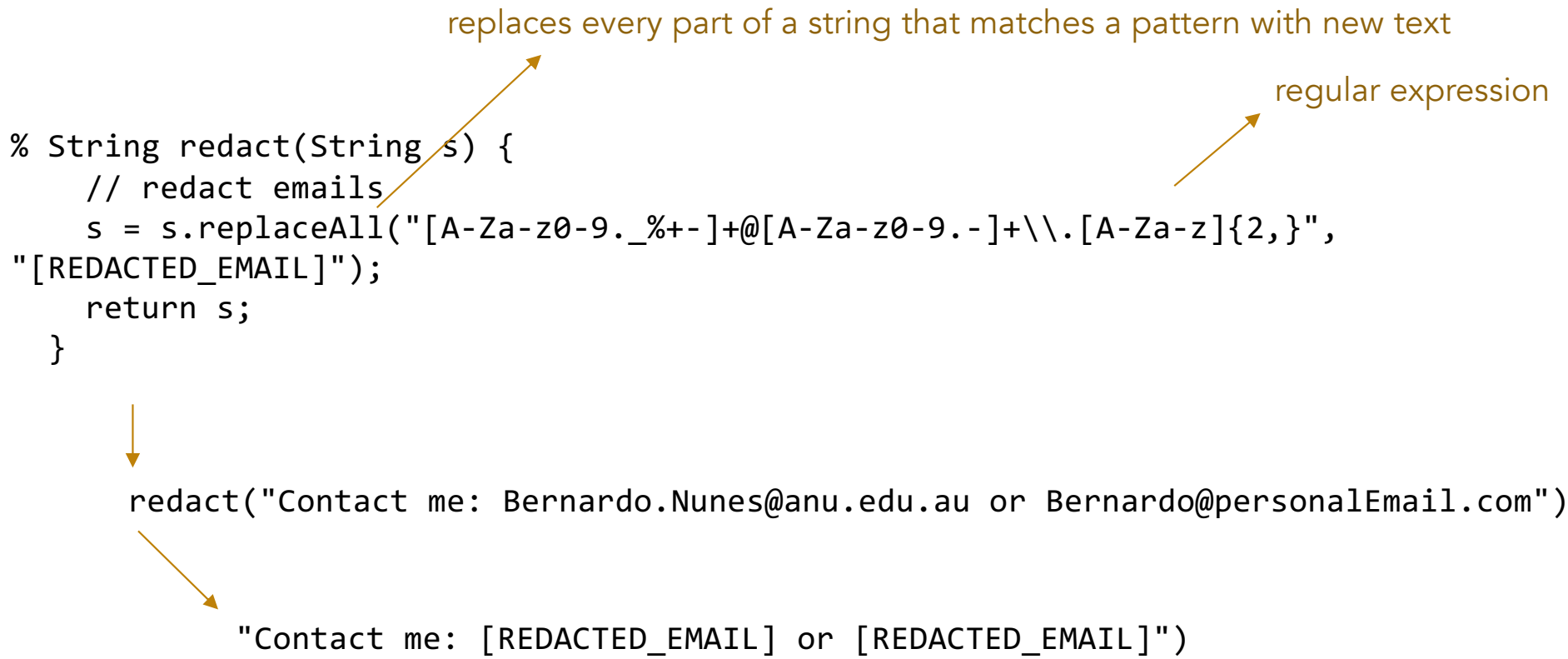
```
jshell> /save mysnippets.jsh //saves the current active snippets to mysnippets.jsh
jshell> /save -history myhistory.jsh // saves the history of all of the snippets and
commands (including invalid ones)
```

More about JSHELL commands at
https://docs.oracle.com/en/java/javase/25/docs/specs/man/jshell.html

<JSHELL FUN TIME>

creating small utility tools

Using JShell for quick, practical utilities



Using JShell for quick, practical utilities

```
import java.awt.Toolkit;

% void focus(int minutes) throws Exception {
    System.out.println("Focus started for " + minutes + " minute(s)");
    Thread.sleep(minutes * 60 * 1000);
    Toolkit.getDefaultToolkit().beep();   // beep
    System.out.println("Time's up! Take a break");
}
```

```
focus(1); // demo with 1 minute
```


Using JShell for quick, practical utilities

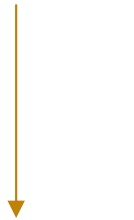
```
import java.time.*;

% String until(int y,int m,int d,int h,int min){
    var dur = Duration.between(LocalDateTime.now(), LocalDateTime.of(y,m,d,h,min));
    if (dur.isNegative()) return "Passed";
    long tm = dur.toMinutes();
    return (tm/1440)+"d "+((tm%1440)/60)+"h "+(tm%60)+"m";
}
```

```
until(2026,3,25,23,5)
```

```
22d 10h 5m
```


Using JShell for quick, practical utilities

```
% int roll() {
    return 1 + new java.util.Random().nextInt(6);
}
```



```
roll()
```

```
6
```


<Exercises>

Which of the following is **true**?

A. The JVM is platform-independent
B. Java bytecode is platform-dependent
C. The JVM is platform-dependent
D. Java source code runs directly on hardware

<Exercises>

You have a Mac with an Apple M2 processor.
Which JDK should you download?

A. x86
B. x64
C. ARM64
D. Any, they are identical

<Exercises>

In older Java versions, code was compiled once using _____ and run multiple times using _____.

<Exercises>

Which of the following is **NOT** a typical responsibility of an IDE?
A. Syntax highlighting
B. Code completion
C. Managing project structure
D. Executing Java bytecode

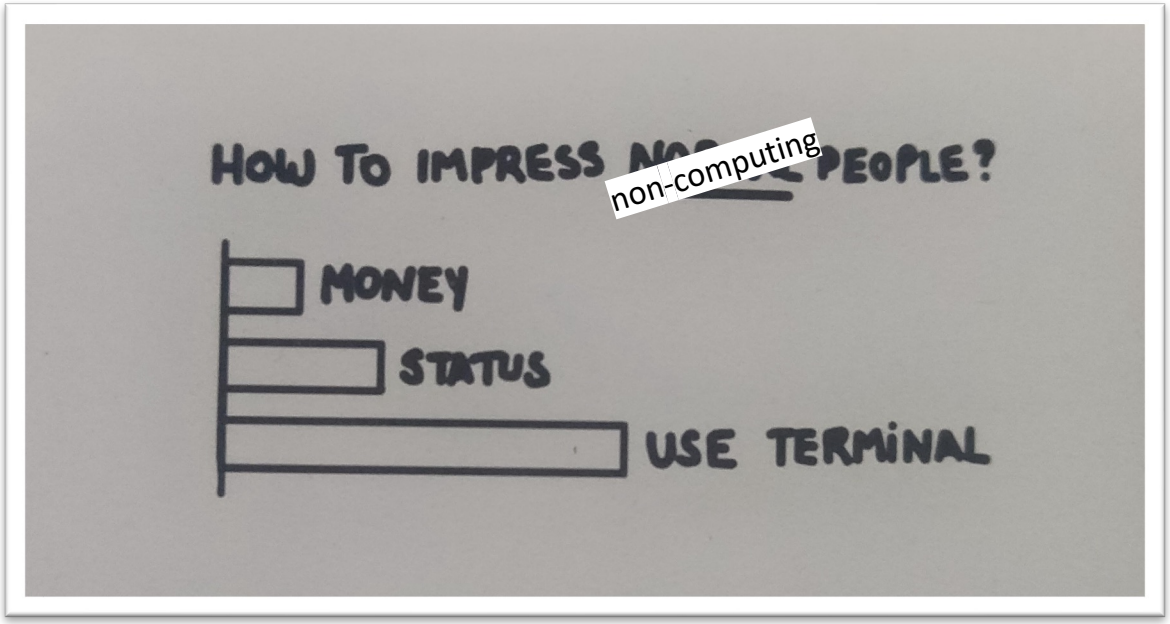
<Exercises>

What is the output in JShell?

3 + 9
5 + $1

A. 14
B. 17
C. Error
D. 12

MEME



Thank you

Australian National University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C